

PREPARATION REPORT LAB1

ADVANCED CPU ARCHITECTURE AND HARDWARE

ACCELERATORS LAB

361.1.4693

Yoav Tsur 211903406

Elad Lavi 211626551

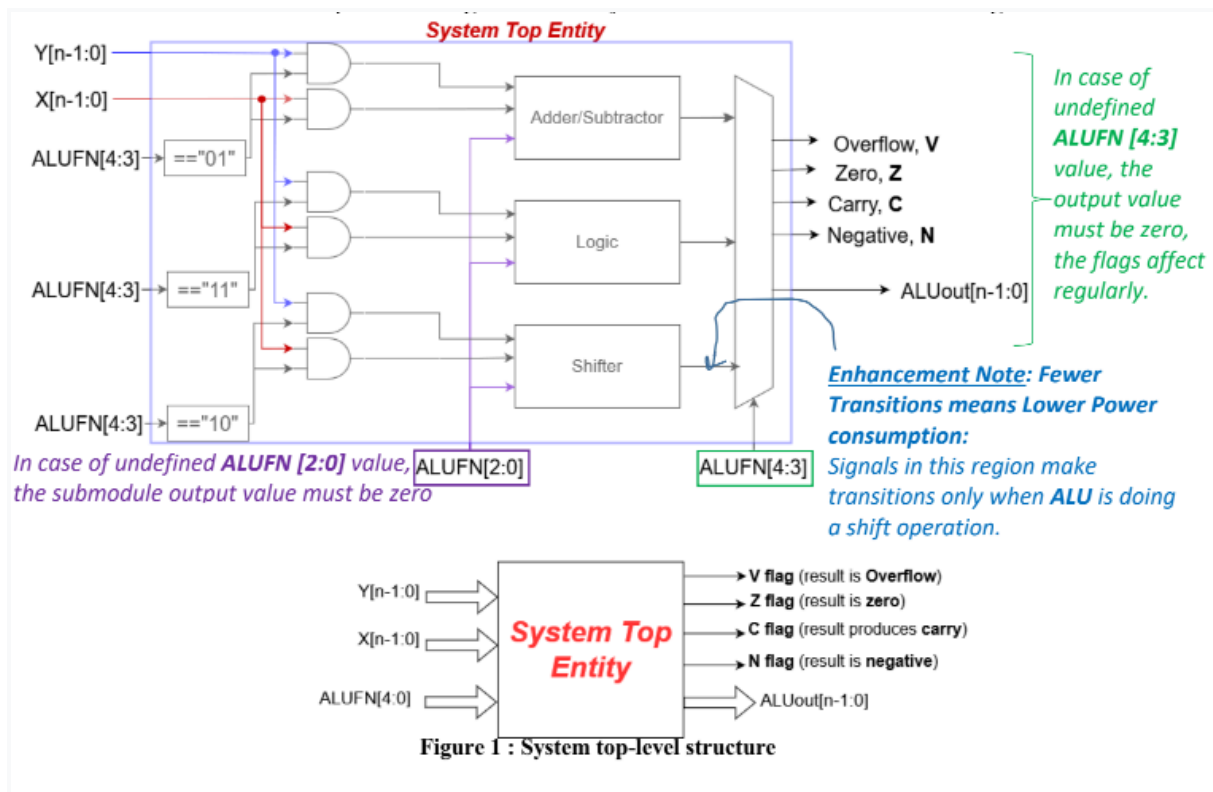
2026 Semester B

מבוא

במעבדה זו חווינו התנסות ראשונה עם שפת תיאור החומרה VHDL. התעסקנו עם Code Structure, Data Types, Operators and Attributes, Concurrent Code, Design Hierarchy, Packages and Components. במעבדה זו, תכננו מודול המכיל את שלושת תת-המודולים הבאים:

- מודול מחבר/חיסור גנרי בין שני וקטורים Y ו- X בגודל n סיביות.
- מודול גנרי המבוסס על גודל n סיביות של Barrel-Shifter (ברירת המחדל היא $n=8$).
- לוגיקה בוליאנית הפועלת על וקטורים X ו- Y לפי סיביות.

להלן המערכת שנרצה לממש:



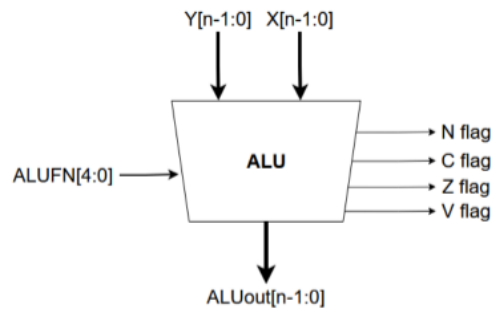
המערכת שלנו למעשה מקבלת 3 כניסות:

- וקטור X
- וקטור Y
- $ALUFN$ שזהו וקטור באורך 5 שקובע את סוג הפעולה שמתבצעת.

כאשר $ALUFN[4:3]$ שווה ל- "01" המודול הנבחר הוא מודול AdderSuber.
 כאשר $ALUFN[4:3]$ שווה ל- "11" המודול הנבחר הוא מודול Logic.
 כאשר $ALUFN[4:3]$ שווה ל- "10" המודול הנבחר הוא מודול Shifter.

בנוסף כל מודל יכול לבצע מספר פעולות שהן תלויות ב- $ALUFN[2:0]$. המערכת שלנו מוציאה :

- וקטור ALUOUT שזהו וקטור תוצאה בין וקטור X לוקטור Y לפי הפעולה שנבחרה לעשות
 - 4 דגלים, N,Z,C,V, כאשר N מסמן אם התוצאה שלילית, Z אם התוצאה היא 0, C אם יש Carry ו-V אם קיים Overflow בתוצאה (מקרה זה רלוונטי רק עבור המודול ADDER/SUB).
- בנוסף כאשר ALUFN יהיה ערך לא מוגדר נדאג שהתוצאה תהיה מיידית 0 ובהתאם לכך הדגלים.



להלן ה-ISA של המערכת:

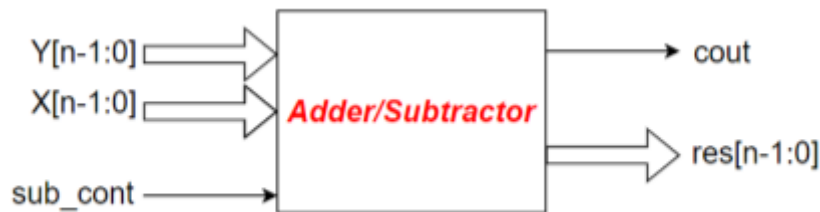
Function Kind	Decimal value	ALUFN	Operation	Note
Arithmetic	8	01000	$Res=Y+X$	Simple addition
	9	01001	$Res=Y-X$	Used also for comparison operation
	10	01010	$Res=neg(X)$	The 2's complement of X
	11	01011	$Res=Y+2$	Double increment of Y
	12	01100	$Res=Y-2$	Double decrement of Y in one
Shift	16	10000	$Res=SHL\ Y,X(k-1\ to\ 0)$	Shift Left Y of $q \triangleq X(k-1 \dots 0)$ times $Res=Y(n-1-q \dots 0) \# (q@0)$ When $k = \log_2 n$
	17	10001	$Res=SHR\ Y,X(k-1\ to\ 0)$	Shift Right Y of $q \triangleq X(k-1 \dots 0)$ times $Res=(q@0) \# Y(n-1 \dots q)$ When $k = \log_2 n$
Boolean	24	11000	$Res=not(Y)$	
	25	11001	$Res=Y\ or\ X$	
	26	11010	$Res=Y\ and\ X$	
	27	11011	$Res=Y\ xor\ X$	
	28	11100	$Res=Y\ nor\ X$	
	29	11101	$Res=Y\ nand\ X$	
	30	11110	$Res=Y\ xnor\ X$	

Table 1: Selected operations

מודול AdderSub

הסבר תיאורטי:

מודול זה הינו אחד משלושת המודולים בשכבה התחתונה ותפקידו הוא לבצע פעולות אריתמטיות הנקבעות לפי ALUFN_1 (שלושת הביטים LSB שלו). המודול מבצע את כל חמשת הפעולות ע"י שימוש במחבר אדווה הממומש באמצעות FA בסיסי ומוציא כתוצאה את פלט הפעולה, סיבית נשא

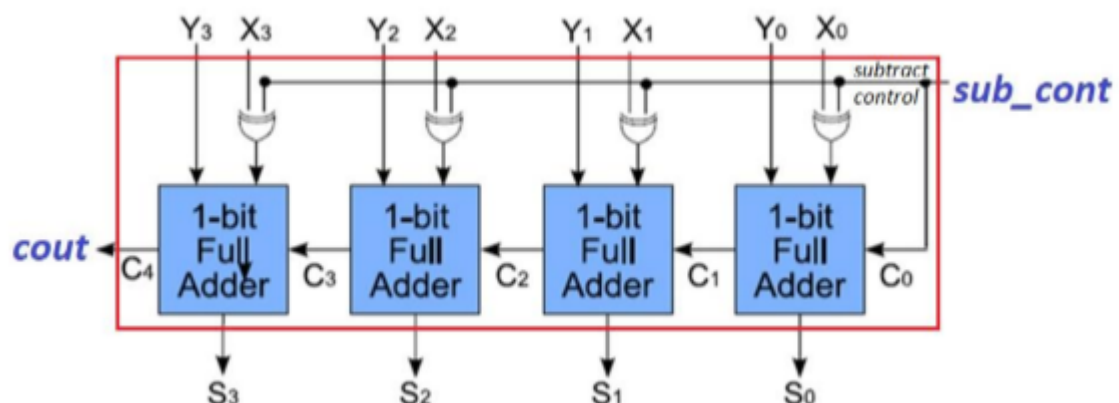


מצורף באיור הבא:

נשים לב כי מוגדר בכניסה למחבר האדווה גם קו בקרה SUB_CONT. מתוך הסתכלות על שלושת הסיביות LSB של ALUFN_1 נראה כי עבור פעולות חיבור נרצה כי קו הבקרה יהיה 0 ועבור פעולות חיסור או משלים נרצה שקו הבקרה יהיה '1'. זה הגיוני כיוון שעבור פעולות חיסור במספרים SIGNED נבצע את הפעולה ע"י שימוש בשיטת המשלים ל-2. כך שאנו הפוכים את הסיביות של הוקטור X ומוסיפים בחיבור הראשון $C_{in} = 1$. את הקו הבקרה קבענו לפי XOR של שלושת הסיביות כיוון שעבור מספר '1' אי זוגי תוצאת ה XOR הינה 1 ועבור מספר '1' זוגי תוצאת ה XOR הינה 0 וזה מתאים במדויק עבור המקרים שלנו כפי שמצורף בתמונה הבאה מתוך ההוראות.

Arithmetic	8	01000	Res=Y+X	Simple addition
	9	01001	Res=Y-X	Used also for comparison operation
	10	01010	Res=neg(X)	The 2's complement of X
	11	01011	Res=Y+2	Double increment of Y
	12	01100	Res=Y-2	Double decrement of Y in one

את מימוש כל הפעולות במודל זה ביצענו ע"י לולאה שמשתמשת בסינגל reg מסוג וקטור של ביטים באורך n עבור שמירת ערך ה C_{in} וה C_{out} בצורה גנרית לכל גודל של וקטור. מצורף, איור הממחיש את דרך פעולת מחבר האדווה עבור הפעולות האריתמטיות נרצה לבצע:



נפרט על הפעולות:

פעולת simple addition: במקרה זה אנו מחשבים את תוצאת החיבור של $Y+X$ ולכן נשתמש במחבר האדווה בצורה ישירה ונשים לב כי קו הבקרה 0 אינו משנה את הוקטור X כיוון שXOR עם 0 משאיר וקטור כמו שהוא.

פעולת subtracting: במקרה זה אנו מבצעים חיסור ע"י $Y-X$. בפעולה זו קו הבקרה הינו 1 והוא משמש כמהפך ע"י שער XOR לוקטור X וגם ככניסת סיבית Cin לחיבור הראשון של מחבר האדווה וכך אנחנו

מממשים לפי הגדרה את פעולת החיסור במספרים signed.

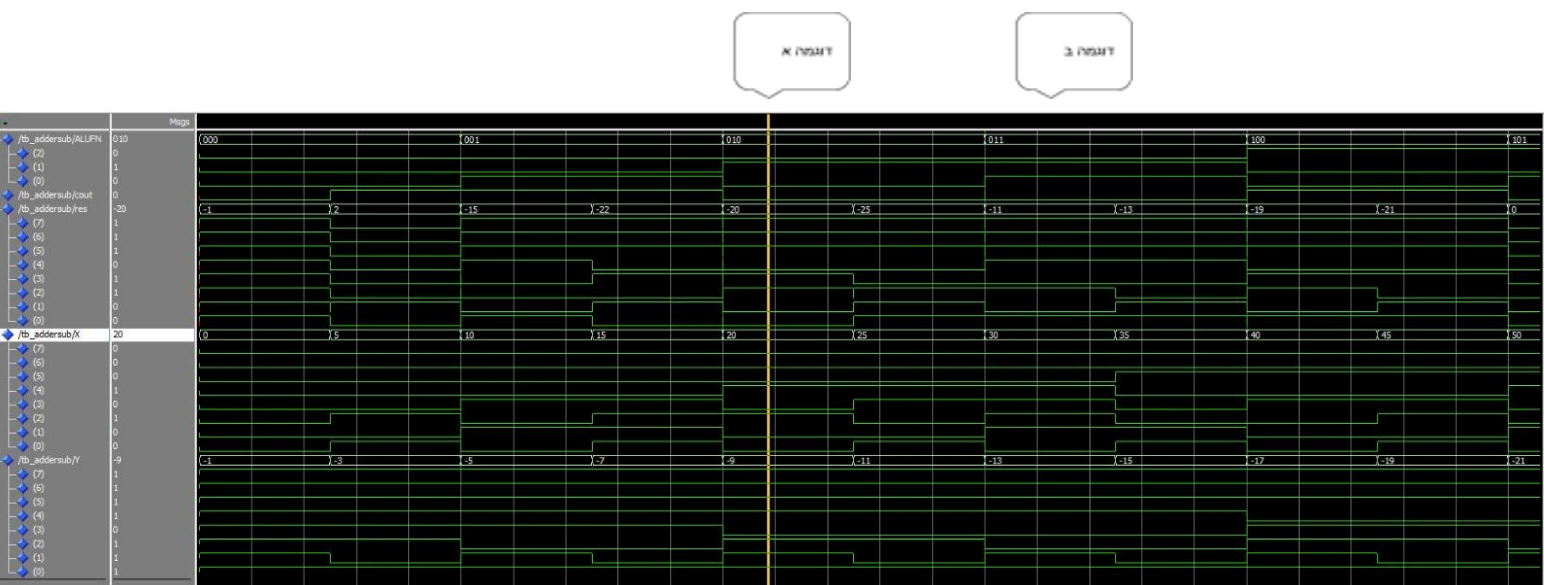
פעולת Negative: במקרה זה נרצה לקחת את וקטור הכניסה X ולהחזיר בתוצאה וקטור שערכו יהיה $-X$.

נממש זאת בדומה לחיסור רגיל ע"י שיטת המשלים ל2 כאשר נגדיר את וקטור הכניסה Y למחבר האדווה להיות וקטור שכולו אפסים.

פעולת double increment: בפעולה זו נרצה לממש את הפעולה $Y+2$ ונוכל לעשות זאת כמו ע"י חיבור רגיל כאשר נגדיר את הוקטור X בכניסה למחבר האדווה להיות בעל ערך '2' בצורה בינארית. פעולת double decrement: כמו בפעולת החיבור כאשר נגדיר את הוקטור X להיות בעל הערך '2' נקבל את תוצאה $Y-2$.

תוצאות הסימולציה:

כעת נרצה להריץ סימולציה עבור המודול הזה שתכלול מספר מקרים בסיסיים יחד עם מקרי קצה שיכולים להתקיים כך שבכל אחד מהמקרים ביצענו השוואה בין מה שהתקבל לבין התוצאה הנכונה שחושבה על ידינו המקרים שנלקחו בחשבון הם:
חיבור עם וללא carry, חיסור, היפוך, כניסה ALU לא מוגדר.
להלן תוצאות הסימולציה בתצוגת DEC:



ניתן לראות היכן שהחץ הצהוב כי הALU מוגדר להיות 010 שזוהי פעולת NEG ואכן ניתן לראות כי X הוא 20 ואילו התוצאה היא -20.

מקרה נוסף לצורך הדוגמה: כאשר הALU הוא 011 הפעולה היא $Y+2$ ואכן ניתן לראות כאשר $Y=-13$ התוצאה היא -11.

מודול SHIFTER:

הסבר תיאורטי:

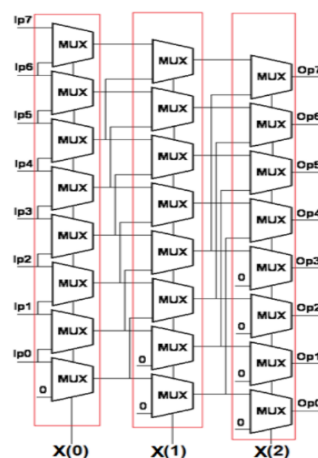
מודול זה הינו המודול השני מתוך שלושת המודולים בשכבה התחתונה ותפקידו לבצע שתי פעולות אפשריות כתלות בשלושת סיביות ה LSB של וקטור הכניסה ALUFN_I. הפעולות האריתמטיות נרצה לבצע הינן הזזה ימינה והזזה שמאלה של וקטור הכניסה Y כאשר אנחנו בוחרים את כמות ההזזות כתלות בא הסיביות ה LSB של וקטור הכניסה X. כיוון ההזזה נקבע ע"י ה LSB של וקטור הכניסה ALUFN_I כאשר '0' מסמן הזזה שמאלה ו'1' מסמן הזזה ימינה. כאשר אנחנו מזיזים את הוקטור Y שמאלה אנחנו משרשרים אפסים בצד ימין כ-כמות ההזזות. באופן סימטרי עבור ההזזות ימינה. מצורף תיאור גרפי של המודול:



הסבר אופן הפעולה: על מנת לממש את השיטת הגדרנו סיגנל MATRIX 1Dx1D בגודל K+1 (המקום ה-0 זה וקטור Y לפני ההזזות) כאשר כל תא במטריצה מייצג את הוקטור Y לאחר ההזזה ה-i מתוך K ההזזות.

בנוסף הגדרנו וקטור המשמש כזיכרון לתוצאת carry. הלולאה פועלת בצורה הבאה: בכל שלב אנחנו מזיזים את הוקטור Y שעבר i-1 שלבי ההזזות כמות של i^2 הזזות ימינה/שמאלה שאלין משורשרים אפסים כ-כמות ההזזות וזאת כתלות בביט ה LSB של הוקטור X. אם הביט הנ"ל הוא 0 אין צורך להזיז בכלל את הוקטור ונוכל להעתיקו באופן זהה למקום הבא במטריצה. אם הביט הוא '1' אזי נבצע את ההזזות. בנוסף אנחנו כותבים לוקטור carry את הביט האחרון שאותו הוצאנו מתוך וקטור Y.

בסיום הלולאה נקבל את הוקטור שעבר את מספר ההזזות כתלות בערך הדצימלי של K הסיביות ה LSB של X ואת carry. מצורף איור הממחיש את פעולת ההזזה:

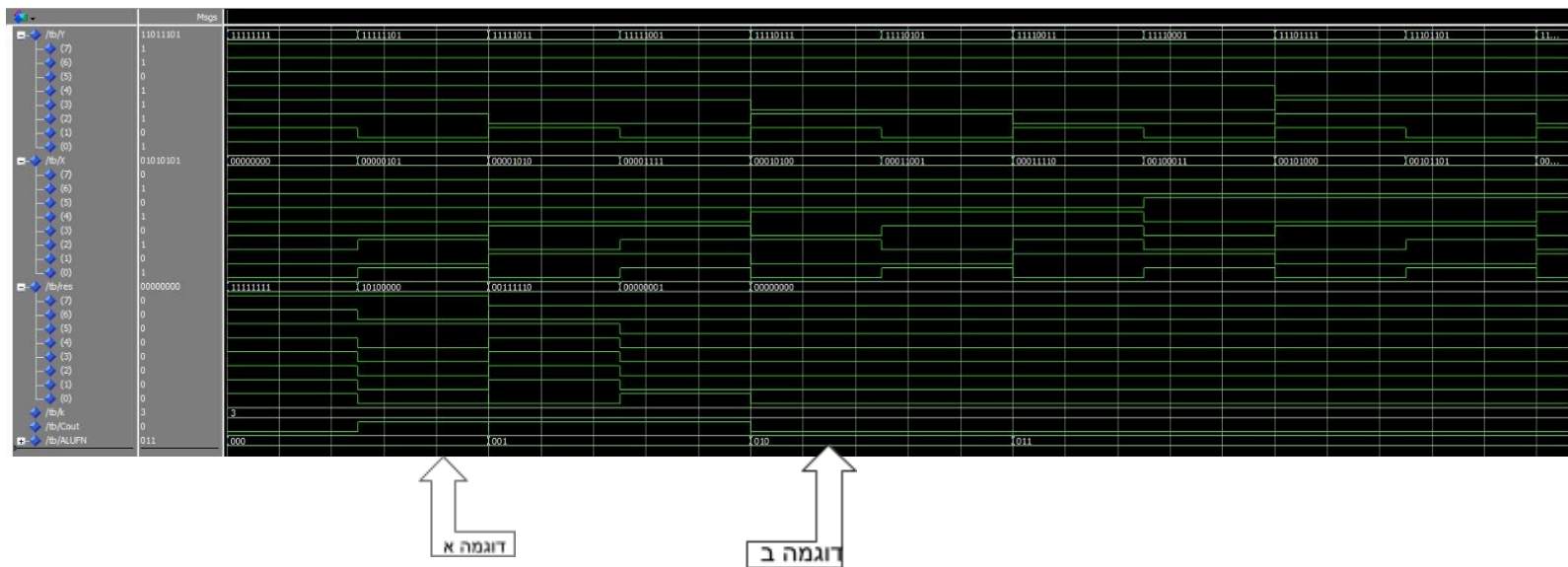


תוצאות הסימולציה:

כעת נרצה להריץ סימולציה עבור המודול הזה שתכלול מספר מקרים בסיסיים יחד עם מקרי קצה שיכולים להתקיים כך שבכל אחד מהמקרים ביצענו השוואה בין מה שהתקבל לבין התוצאה הנכונה שחושבה על ידינו המקרים שנלקחו בחשבון הם :

הזזה ימינה, הזזה שמאלה וכניסה ALU לא מוגדרת.

להלן תוצאות הסימולציה בתצוגה בינארית:



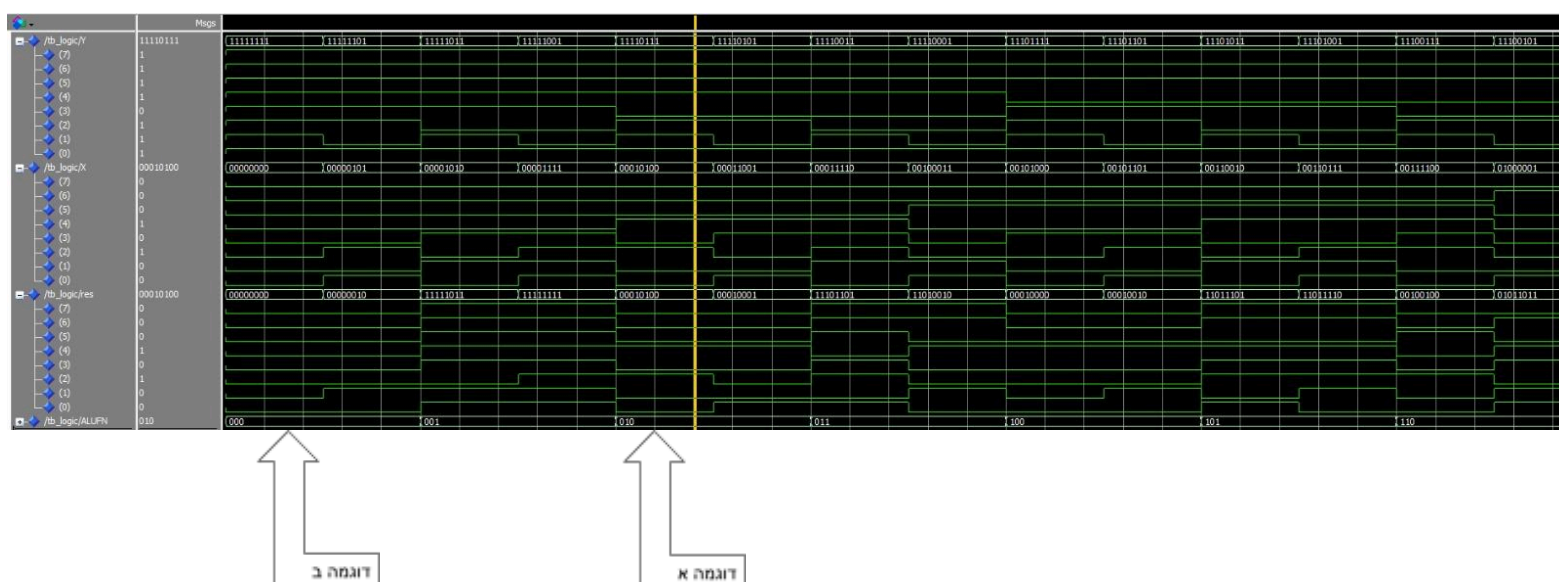
ניתן לראות כי במקרה הראשון למשל כי עבור ALU 000 בוצעה הזזה שמאלה לוקטור Y. הוקטור Y הינו 11111101. ואילו $X \ll 3 = K$ ב 3 הסיביות האלו : 101 נבע 5 הזזות שמאלה ולכן וקטור התוצאה הינו : 10100000, בנוסף ה-COUT עלה ל 1 כי יש carry. דוגמה נוספת עבור ALU לא מוגדר ניתן לראות כי וקטור התוצאה הוא וקטור אפסים וה-COUT .

מודול LOGIC:

מודול זה הינו המודול השלישי בשכבה התחתונה ותפקידו לבצע כ7 פעולות לוגיות מובנות בשפת VHDL בין הוקטורים X ו Y כתלות בשלושת הביטים ה LSB של וקטור הכניסה ALUFN_I. מימשנו את המודול באמצעות MUX המשתמש ב-3 ביטי הבקרה כדי לבחור אחת מתוך 7 פעולות שונות כאשר הכניסה השמינית מנוצלת למניעת מצבים לא מוגדרים וזאת ע"י איפוס המוצא.

תוצאות הסימולציה:

כעת נרצה להריץ סימולציה עבור המודול הזה שתכלול מספר מקרים בסיסיים יחד עם מקרי קצה שיכולים להתקיים כך שבכל אחד מהמקרים ביצענו השוואה בין מה שהתקבל לבין התוצאה הנכונה שחושבה על ידינו המקרים שנלקחו בחשבון הם: כל אחת מהפעולות הקיימות ב ISA ובנוסף ALU לא תקין. להלן תוצאות הסימולציה בתצוגת בינארית:



ניתן לראות היכן שהקו הצהוב כי עבור $ALU=010$ מוגדרת הפעולה AND. ואכן עבור $X=00010100$ ו $Y=11110111$ התוצאה היא : 00010100 .
דוגמה נוספת כאשר ה ALU 000 ביצענו NOT Y ואכן עבור $Y=11111101$ התוצאה הינה: 00000010 .

מודול TOP:

הסבר תיאורטי:

מודול זה הינו המודול אשר עוטר את המערכת כולה. הוא מקבל את וקטורי הכניסה X, Y ואת וקטור הכניסה $ALUFN_I$ ומעביר לתת-מודול המתאים את הוקטורים X, Y יחד עם שלושת הביטים ה LSB של $ALUFN_I$. בנוסף בחלק זה טיפלו בעניין הדגלים שהמערכת מוציאה כתלות בתוצאה המתקבלת מ-הפעולה המתאימה. על מנת לשמור על צריכת הספק נמוכה נרצה שרק המודל המתאים יבצע את הפעולות ואילו שאר המודולים לא יבצעו פעולות ולכן השתמשנו בשערי AND על מנת להעביר את הוקטורים X, Y רק לסיגנלים הדרושים לביצוע הפעולה (כלומר למודול המתאים). בכל אחד מהפליטים השתמשנו ב MUX ע"י שימוש ב $WITH -SELECT$ כדי להעביר את התוצאה הרלוונטית ל $ALUOUT$ בהתאם לקוד הפעולה.

הסבר עבור הדגלים:

דגל Z : דגל זה שווה ל1 עבור וקטור תוצאה שכולו אפסים, מימשנו את הלוגיקה באמצעות השוואה של וקטור התוצאה לוקטור אפסים שהגדרנו בעצמנו.

דגל C : דגל זה שווה ל1 במידה ויש $carry$, דבר שיכול לקרות רק עבור מודולים $AdderSub$ ו- $Shifter$ ולכן במקרה שאלו היו המודלים דגל C היה שווה ל $carry$ שהוגדר כתוצאה מתת המודול הרלוונטי.

בכל מקרה אחר דגל C שווה ל"0" כיוון שאין $Carry$.

דגל N : דגל זה שווה ל1 במידה וביט ה MSB של התוצאה שווה ל"1" לכן הגדרנו אותו כך באופן ישיר. דגל V : דגל זה מייצג האם פעולת חיבור או חיסור מייצרת "גלישה" כלומר תוצאה שאינה יכולה לקרות וקרתה כתוצאה ממספר מוגבל של ביטים. מצורף התנאים שיכולים לגרום ל- $Overflow$:

In case of ADD operation:

$V=1$:

- Positive + Positive = Negative
 - Negative + Negative = Positive
- otherwise, $V=0$.

In case of SUB operation:

$V=1$:

- Positive - Negative = Negative
 - Negative - Positive = Positive
- otherwise, $V=0$.

ע"י שימוש בלוגיקה בוליאנית, טבלת אמת הגענו למשוואות הבאות היוצרות $Overflow$ כאשר אנחנו מנצלים את ידיעת קוד הפעולה לכך שנוכל להתחשב במקרי גלישה בפעולות חיבור וחיסור בנפרד:

```
v_adder <= (X_i(n-1) xor s_addersub(n-1)) and (y_i(n-1) xor s_addersub(n-1));  
v_subber <= (X_i(n-1) xor y_i(n-1)) and (y_i(n-1) xor s_addersub(n-1));
```

תוצאות הסימולציה:

כעת נרצה להריץ את הסימולציה עבור המערכת הכוללת שניתנה לנו דרך המודל vhd 1.ref tb
להלן תוצאות הסימולציה:

File Edit Search Options Actions Help		C:\Users\yves\Downloads\GSM-20263221017502-1-0516MB_refl.mt	
C:\Users\yves\Downloads\GSM-20263221017502-1-0516MB_refl.mt		C:\Users\yves\Downloads\GSM-20263221017502-1-0516MB_refl.mt	
1	ps	ps	ps
2	delta	delta	delta
3	/cb_refi/Y	/cb/Y	/cb/ALDout
4	/cb_refi/X	/cb/RIflag	/cb/RIflag
5	/cb_refi/ALDout	/cb/ALDout	/cb/RIflag
6	/cb_refi/RIflag	/cb/RIflag	/cb/RIflag
7	0 +9	0 +8	0 +8
8	50000 +9	50000 +7	50000 +7
9	100000 +10	100000 +8	100000 +8
10	150000 +8	150000 +6	150000 +6
11	200000 +10	200000 +8	200000 +8
12	250000 +8	250000 +6	250000 +6
13	300000 +10	300000 +8	300000 +8
14	350000 +7	350000 +7	350000 +7
15	400000 +9	400000 +8	400000 +8
16	450000 +7	450000 +8	450000 +8
17	500000 +11	500000 +11	500000 +11
18	550000 +10	550000 +9	550000 +9
19	600000 +9	600000 +7	600000 +7
20	650000 +11	650000 +9	650000 +9
21	700000 +8	700000 +7	700000 +7
22	750000 +11	750000 +11	750000 +11
23	800000 +10	800000 +9	800000 +9
24	850000 +8	850000 +8	850000 +8
25	900000 +8	900000 +7	900000 +7
26	950000 +0	950000 +0	950000 +0
27	1000000 +8	1000000 +8	1000000 +8
28	1050000 +1	1050000 +1	1050000 +1
29	1100000 +9	1100000 +9	1100000 +9
30	1150000 +8	1150000 +8	1150000 +8
31	1200000 +8	1200000 +7	1200000 +7
32	1250000 +7	1250000 +7	1250000 +7
33	1300000 +8	1300000 +8	1300000 +8
34	1350000 +1	1350000 +1	1350000 +1
35	1400000 +6	1400000 +6	1400000 +6
36	1450000 +5	1450000 +5	1450000 +5
37	1500000 +5	1500000 +5	1500000 +5
38	1550000 +5	1550000 +5	1550000 +5
39	1600000 +5	1600000 +5	1600000 +5
40	1650000 +5	1650000 +5	1650000 +5
41	1700000 +5	1700000 +5	1700000 +5
42	1750000 +5	1750000 +5	1750000 +5
43	1800000 +5	1800000 +5	1800000 +5
44	1850000 +3	1850000 +3	1850000 +3
45	1900000 +3	1900000 +3	1900000 +3
46	1950000 +1	1950000 +1	1950000 +1
47	2000000 +1	2000000 +1	2000000 +1

לאחר הרצת TB שניתן לנו מראש ייצרנו קובץ LST , את הקובץ הזה ואת הקובץ LST שניתן לנו מראש השווינו בתוכנה TextDiff לצורך מציאת הבדלים.

מאחר והכל ירוק ניתן להבין כי ההרצות אכן זהות ולכן ניתן לראות כי עבור TB שנכתב לנו מראש עבור בדיקת מקרים מסוימים הקוד שלנו עובד כנדרש.

בנוסף הנה קובץ WAVE שבו ניתן לראות את תוצאות הסימולציה לTB שאותו אנחנו כתבנו, הוא מכיל

למעשה מקרים מכל המודלים האפשריים הכולל מקרי קצה.

		Rflag																				
7	11111101	11111111	11111110	11111011	11111001	11110111	11110101	11110011	11110001	11101111	11101101	11101011	11101001	11100111	11100101	11100011	11100001	11011111	11011101	11011011	11011001	11010111
6	1																					
5	1																					
4	1																					
3	1																					
2	1																					
1	0																					
0	1																					
7	00000100	11111111	00000100	00001001	00001110	00010011	00011000	00011101	00100010	00100111	00101100	00110001	00110110	00111011	01000000	01000101	01001010	01001111	01010100	01011001	01011110	01100011
6	0																					
5	0																					
4	0																					
3	0																					
2	1																					
1	0																					
0	0																					
7	8	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	
6	0																					
5	0																					
4	0																					
3	0																					
2	0																					
1	0																					
0	0																					
7	00000001	11111110	00000000	11110010	11101011	11101101	11101000	11110101	11110011	11101101	11101011	00011100	00011111	10101100	10100101	00000000	00000000	00000000	00000000	00110100	00110111	11011001
6	0																					
5	0																					
4	0																					
3	0																					
2	0																					
1	0																					
0	1																					
7	1																					
6	1																					
5	1																					
4	1																					
3	1																					
2	1																					
1	0																					
0	0																					

ניתן לראות בתמונה את תוצאות הסימולציה.
למשל היכן שהחץ הצהוב הALU בתצורת DEC הוא 8 שזה למעשה 01000 שזהו מודל Addsub פעולת חיבור בין הוקטורים X ו Y.
לכן, עבור $X=00000100$ כלומר 4 ועבור $Y=11111101$ כלומר 3- בשיטת משלים ל 2 התוצאה הינה 1 ובצורת ביטים ניתן לראות כי הוקטור הוא 00000001.
בנוסף ניתן לראות בתמונה כי הורם דגל Carry.